XCPC Cheat Sheet

brofea

2026.3.20 Update

目录

1. 技巧和 STL . . . 3
   1.1. 环境配置 . . . 3
   1.2. 基本语法 . . . 3
   1.3. STL 函数 . . . 4
   1.4. STL 容器和方法 . . . 5
   1.5. 复杂度 . . . 6
2. 基础算法 . . . 6
   2.1. 排序 . . . 6
   2.2. 二分查找 . . . 7
   2.3. 高精度 . . . 7
   2.4. 位运算 . . . 8
   2.5. 贪心 . . . 8
      2.5.1. 区间问题 . . . 8
3. 动态规划 . . . 9
   3.1. 背包问题 . . . 9
   3.2. 序列问题 . . . 10
4. 字符串 . . . 11
   4.1. KMP . . . 11
   4.2. 字典树 Trie . . . 12
   4.3. AC 自动机 . . . 12
   4.4. Manacher . . . 13
5. 数学 . . . 14
   5.1. 质因数 . . . 14
      5.1.1. 质数筛 . . . 14
      5.1.2. LCM & GCD . . . 14
      5.1.3. 质因数分解和因数问题 . . . 15
   5.2. 欧拉函数 . . . 15
      5.2.1. 欧拉定理 . . . 15
      5.2.2. 费马小定理 . . . 16
   5.3. 快速幂和乘法逆元 . . . 17
   5.4. 扩展欧几里得算法 . . . 17
   5.5. 中国剩余定理 . . . 18
   5.6. 线性代数 . . . 18
      5.6.1. 高斯消元 . . . 18
   5.7. 组合数学 . . . 19
      5.7.1. 递推法 . . . 19
      5.7.2. 预处理法 . . . 19
      5.7.3. Lucas 定理 . . . 19
      5.7.4. 分解质因数 . . . 20
   5.8. 卡特兰数 . . . 21
   5.9. 简单博弈论 . . . 21
      5.9.1. Nim 游戏 . . . 21

6. 数据结构 21
6.1. 并查集 21
6.2. ST 表 22
6.3. 堆 22
6.4. 树状数组 23
6.5. 线段树 24
6.5.1. 朴素线段树 24
6.5.2. 惰性线段树 28
6.6. 平衡树 30
6.6.1. Treap 30
7. 图论 33
7.1. 基础 33
7.2. 最短路 33
7.2.1. 朴素 Dijkstra 34
7.2.2. 堆优化 Dijkstra 35
7.2.3. Bellman-Ford 36
7.2.4. SPFA 37
7.2.5. Floyd-Warshall 37
7.3. 最小生成树 38
7.3.1. Prim 38
7.3.2. Kruskal 38
7.4. 二分图 39
7.4.1. 染色法 39
7.4.2. 匈牙利算法 40
7.5. 联通性 40
7.5.1. Tarjan 40

1. 技巧和 STL

1.1. 环境配置

VS Code 配置

1. 安装扩展：Chinese (Simplified)、C/C++、C/C++ Extension Pack、Code Runner、cph，选装：CodeLLDB、clangd
2. 设置编译和调试选项
   - 创建新 task.json 并在 arg 中加入 `"-std=c++17"` 和 `"-O2"`
   - 设置 Code Runner：启用 Run in Terminal 和 Save File Before Run
   - 设置 Code Runner：在 Excuter Map 中 cpp 键中添加 `-std=c++17 -O2`
   - 设置 cph：在 Cpp: Args 中添加 `-std=c++17 -O2`

1.2. 基本语法

宏、关流和别名

```
#define int long long                       // 小心使用
#define endl '\n'

#define PII pair<int, int>
#define uset unordered_set
#define umap unordered_map
#define iter iterator
using uint = unsigned int;
using ll = long long;

ios::sync_with_stdio(0);                    // 关闭 C++ & C 标准流同步
cin.tie(NULL);                              // 取消 cin 和其他流绑定
cout.tie(NULL);                             // 取消 cout 和其他流绑定
```

内建函数和常量

```
__builtin_popcount(x)       // 计算 x 中 1 的个数
__builtin_ctz(x)            // 计算 x 末尾零的个数
__builtin_clz(x)            // 计算 x 前导零的个数，可用于获得最高位 1 的位置
__builtin_parity(x)         // 计算 x 中 1 的个数的奇偶性（返回 1 或 0）
__builtin_ffs(x)            // 返回 x 最低位的 1 的位置（从 1 开始计数）
                            // 上述函数在函数名后加 ll 即为 long long 版本
__INT_MAX__                 // int 最大值 2147483647
__LONG_LONG_MAX__           // long long 最大值 9223372036854775807
                            // min 通常为 -max-1，如:
#define __INT_MIN__ (-__INT_MAX__ - 1)
```

初始化数组

```
int arr[100];
vector<int> vec(100);

fill(arr, arr + 100, 1);            // 把所有元素设为 1
fill(vec.begin(), vec.end(), 1);    // 把所有元素设为 1
memset(arr, 0, sizeof(arr));        // 把所有字节设为 0
memset(arr, 0x3f, sizeof(arr));     // 把所有字节设为 0x3f3f3f3f
memset(arr, -1, sizeof(arr));       // 把所有字节设为 0xffffffff（仅限 int、short、ll）
```

输入输出

```
cout << setfill('0') << left << setw(5) << a;   // 带填充的左对齐固定宽度（默认右）
cout << fixed << setprecision(3) << f;          // 设置小数点位数

cin >> n
cin.ignore();                                   // cin 后使用 getline
getline(cin, s);
scanf("%[^\n]", str);                           // 忽略空格输入字符数组
```

1.3. STL 函数

常量和数学函数

```
INT_MAX                                         // 2147483647
INT_MIN                                         // -2147483648
LLONG_MAX                                       // 9223372036854775807
LLONG_MIN                                       // -9223372036854775808
DBL_MAX                                         // 1.79769e+308
DBL_MIN                                         // 2.22507e-308
M_PI                                            // 3.14159265358979323846
powl(a, b)                                      // pow 的 long double 版本
round(p, (1.0/n))                               // p 的 n 次方根
```

非方法 STL 函数

```
shuffle(v.begin(), v.end(), std::default_random_engine());  // 随机打乱
copy(v.begin(), v.end(), dest.begin());                     // 复制区间内的元素到 dest
swap_ranges(v1.begin(), v1.end(), v2.begin());              // 交换等长区间内的元素
reverse(v.begin(), v.end());                                // 反转区间内的元素
replace(v.begin(), v.end(), a, b);                          // 替换区间内所有 a 为 b
replace_if(v.begin(), v.end(), pred, b);    // 替换满足 pred 条件的值为 b, pred 是 bool
函数

merge(a.begin(), a.end(), b.begin(), b.end(), result.begin());      // 合并两个有序区间
set_union(a.begin(), a.end(), b.begin(), b.end(), result.begin());          // 取并集
set_intersection(a.begin(), a.end(), b.begin(), b.end(), result.begin());   // 取交集
set_difference(a.begin(), a.end(), b.begin(), b.end(), result.begin());     // 取差集
// 上述四个函数都可在末尾增加 comp 参数, comp 是一个 bool 类型的比较函数

sort(v.begin(), v.end());                        // 升序
sort(v.begin(), v.end(), greater<int>());        // 降序
sort(v.begin(), v.end(), cmp);                   // 自定义排序, cmp 是一个 bool 类型的比
较函数
stable_sort();              // 稳定排序, 使用归并排序实现 O(nlogn), 用法同 sort
partial_sort(v.begin(), v.begin() + k, v.end()); // 将前 k 小的元素放在前面 O(nlogk)
is_sorted(v.begin(), v.end());                   // 判断是否为升序
is_sorted(w.begin(), w.end(), greater<int>());   // 判断是否为降序
is_sorted_until(v.begin(), v.end());             // 返回第一个不满足升序的迭代器

next_permutation(a.begin(), a.end()); // 下一个字典序排列, 如果已是最大排列则返回 false
O(n)
prev_permutation(a.begin(), a.end()); // 上一个字典序排列, 如果已是最小排列则返回 false
O(n)

auto it = find(v.begin(), v.end(), k);           // 查找值为 k 的元素, 返回迭代器
auto it = find_if(v.begin(), v.end(), pred);          // 查找满足 pred 条件的元素, 返回迭
代器
```

```
it2 - it1                           // 计算两个迭代器之间的距离，仅用于 vector, deque,
array
distance(it1, it2);                 // 计算两个迭代器之间的距离，可用于 list, set, map

*max_element(a, a + n)              // 返回数组最大值
*max_element(a.begin(), a.end())    // 返回vector最大值
// min_element 用法同上

// 排序后去重
sort(a.begin(), a.end());
a.erase(unique(a.begin(), a.end()), a.end());
// C风格排序后去重
sort(a, a + n);
n = unique(a, a + n) - a;
```

1.4. STL 容器和方法

字符串

```
uint find(string &s2, uint pos = 0);                // 正序查找子串
uint rfind(string &s2, uint pos = end);             // 反序查找子串
uint find_first_of(string &s2, uint pos = 0);       // 正序查找在 s2 中的字符
uint find_last_of(string &s2, uint pos = end);      // 反序查找在 s2 中的字符
uint find_first_not_of(string &s2, uint pos = 0);   // 正序查找不在 s2 中的字符
uint find_last_not_of(string &s2, uint pos = end);  // 反序查找不在 s2 中的字符
string& insert(uint pos, string &s2);               // 在 pos 位置插入 str
string& substr(uint pos = 0, uint len = npos)       // 返回 pos 开始长度为 len 的子串
string& erase(uint pos = 0, uint len = npos);       // 删除 pos 开始长度为 len 的子串
string& replace(uint pos, uint len1, string &s2);   // 替换 pos 开始长度为 len 的子串
string& insert(uint pos, uint repetitions, char c); // 在 pos 位置插入 repetitions 个 c
// 替换 pos 开始长度为 len 的子串为 repetitions 个 c
string& replace(uint pos, uint len1, uint repetitions, char c);
```

增删查改

```
// 查找，注意 list 中为 O(n)
lower_bound(x)              // 返回第一个大于等于 x 的数的迭代器
upper_bound(x)              // 返回第一个大于 x 的数的迭代器

// set / multiset
it++, it--                  // 迭代器支持双向迭代
insert(int x)               // 插入 x，返回 pair<iter, bool>，后者表示插入成功/已经
存在
find(int x)                 // 查找 x，返回迭代器，没找到返回 end()
count(int x)                // 返回 x 的个数
erase(int x)                // 删除所有 x

// map/multimap
insert(PII x)               // 插入键值对 x
erase(PII x)                // 删除键值对 x
find(int x)                 // 查找键 x，返回迭代器，没找到返回 end()
// unordered 系类似，但不支持迭代器和二分查找
```

数据结构

```
priority_queue<int, vector<int>, greater<int>>      // 小根堆
a.erase(unique(a.begin(), a.end()), a.end());       // 数组去重
```

1.5. 复杂度

空间复杂度 Tips

- 64 MB 可以存 1e7 个 int
- 可以输出 `sizeof arr / 1024` 检查数组内存占用，单位是 KB

用数据规模猜算法

$n \le 30$, 指数级别, dfs+剪枝，状态压缩 dp

$n \le 100 \Rightarrow O(n^3)$，floyd，dp，高斯消元

$n \le 1000 \Rightarrow O(n^2)$，$O(n^2 \log n)$，dp，二分，朴素版 Dijkstra、朴素版 Prim、Bellman-Ford

$n \le 10000 \Rightarrow O(n\sqrt{n})$，块状链表、分块、莫队

$n \le 100000 \Rightarrow O(n \log n)$ => 各种 sort，线段树、树状数组、set/map、heap、拓扑排序、dijkstra+heap、prim+heap、Kruskal、spfa、求凸包、求半平面交、二分、CDQ 分治、整体二分、后缀数组、树链剖分、动态树

$n \le 1000000 \Rightarrow O(n)$, 以及常数较小的 $O(n \log n)$ 算法 => 单调队列、 hash、双指针扫描、BFS、并查集，kmp、AC 自动机，常数比较小的 $O(n \log n)$ 的做法：sort、树状数组、heap、dijkstra、spfa

$n \le 10000000 \Rightarrow O(n)$，双指针扫描、kmp、AC 自动机、线性筛素数

$n \le 10^9 \Rightarrow O(\sqrt{n})$，判断质数

$n \le 10^{\{18\}} \Rightarrow O(\log n)$，最大公约数，快速幂，数位 DP

$n \le 10^{\{1000\}} \Rightarrow O(\log^2 n)$，高精度加减乘除

$n \le 10^{\{100000\}} \Rightarrow O(\log k \log \log k)$，k 表示位数，高精度加减、FFT/NTT

2. 基础算法

2.1. 排序

快速排序

```
void quick_sort(int q[], int l, int r)
{
  if (l ≥ r) return;
  int i = l - 1, j = r + 1, x = q[(l + r) >> 1];
  // 随机选取q，用双指针法将将小于q的数放在q的左边，将大于q的数放在q右边
  while (i < j) {
    do i++; while (q[i] < x);
    do j--; while (q[j] > x);
    if (i < j) swap(q[i], q[j]);
  }
  // 递归完成两个区间
  quick_sort(q, l, j), quick_sort(q, j + 1, r);
}
```

归并排序

```
void merge_sort(int q[], int l, int r)
{
  static int tmp[1000000]; // 临时数组
```

```
  if (l ≥ r) return;
  int mid = (l + r) >> 1;
  // 递归成两个区间直到区间长度为1
  merge_sort(q, l, mid);
  merge_sort(q, mid + 1, r);
  // 合并区间
  int k = 0, i = l, j = mid + 1;
  while (i ≤ mid && j ≤ r)
    if (q[i] ≤ q[j]) tmp[k ++ ] = q[i ++ ];
    else tmp[k ++ ] = q[j ++ ];

  while (i ≤ mid) tmp[k ++ ] = q[i ++ ];
  while (j ≤ r) tmp[k ++ ] = q[j ++ ];

  for (i = l, j = 0; i ≤ r; i ++, j ++ ) q[i] = tmp[j];
}
```

2.2. 二分查找

```
// 寻找第一个满足 check 的值
int bsearch_1(int l, int r) {
  while (l < r) {
    int mid = (l + r) >> 1;
    if (check(mid)) r = mid;
    else l = mid + 1;
  }
  return l;
}
// 寻找最后一个满足 check 的值
int bsearch_2(int l, int r) {
  while (l < r) {
    int mid = (l + r + 1) >> 1;
    if (check(mid)) l = mid;
    else r = mid - 1;
  }
  return l;
}
```

2.3. 高精度

```
// 高精加
vector<int> add(vector<int> &A, vector<int> &B) {
  if (A.size() < B.size()) return add(B, A);

  vector<int> C;
  int t = 0;
  for (int i = 0; i < A.size(); i++ ) {
    t += A[i];
    if (i < B.size()) t += B[i];
    C.push_back(t % 10);
    t /= 10;
  }

  if (t) C.push_back(t);
  return C;
}
// 高精减
```

```
vector<int> sub(vector<int> &A, vector<int> &B) {
  vector<int> C;
  for (int i = 0, t = 0; i < A.size(); i++ ) {
    t = A[i] - t;
    if (i < B.size()) t -= B[i];
    C.push_back((t + 10) % 10);
    if (t < 0) t = -1;
    else t = 0;
  }

  while (C.size() > 1 && C.back() == 0) C.pop_back();
  return C;
}
// 高精乘低精
vector<int> mul(vector<int> &A, int b) {
  vector<int> C;

  int t = 0;
  for (int i = 0; i < A.size() || t; i ++ ) {
    if (i < A.size()) t += A[i] * b;
    C.push_back(t % 10);
    t /= 10;
  }

  while (C.size() > 1 && C.back() == 0) C.pop_back();

  return C;
}
// 高精除低精
vector<int> div(vector<int> &A, int b, int &r) {
  vector<int> C;
  r = 0;
  for (int i = A.size() - 1; i >= 0; i -- ) {
    r = r * 10 + A[i];
    C.push_back(r / b);
    r %= b;
  }
  reverse(C.begin(), C.end());
  while (C.size() > 1 && C.back() == 0) C.pop_back();
  return C;
}
```

2.4. 位运算

返回 x 的最低位 1 包括后面的 0

```
int lowbit(int x) {
  return x & -x;
}
```

2.5. 贪心

2.5.1. 区间问题

区间选点

给定 n 个区间，取数量最少的点使得每一个区间内至少有一个点，输出最少点数

1. 将每个区间按右端点从小到大排序
2. 从小到大枚举区间
   1. 如果当前区间已经包含点，跳过
   2. 否则，选择当前区间的右端点

区间选择

给定 n 个区间，要求选择最多的区间使得这些区间两两之间没有交集，求选择的数量最大

1. 将每个区间按右端点从小到大排序
2. 从左到右枚举区间
   1. 如果该区间的左端点大于上一次选择区间的右端点，加入

区间分组

给定 n 个区间，将这些区间分成若干组使得组内部的区间没有交集且组数最少，也就是说有交集的区间一定要分在不同的组内，输出最少组数

1. 将所有区间按左端点从小到大排序
2. 从前往后枚举区间，每次判断是否能放入现有的组（判断该区间左端点是否大于组的右端点）
   1. 如果可以则放入（更新组的右端点）
   2. 如果不行则新开组

区间覆盖

给定 n 个区间和一个额外的区间 $[l, r]$，在 n 个区间中选择尽量少的区间将这个额外区间完全覆盖，输出最少区间数

1. 将区间按照左端点从小到大排序
2. 从前往后枚举区间，在所有能覆盖 l 的区间中选一个右端点最大的，然后将 l 更新为该区间的右端点

3. 动态规划

3.1. 背包问题

01 背包

N 个物品重量 w_i 价值 v_i ,背包总容量 V，每个物品只能选或者不选。$f(i,j)$ 表示选前 i 个物品，背包最大体积为 j 的情况下储存的属性（如最大价值，最小价值，物品数量最多，以下讨论最大价值）

$$f(i,j) = \max(f(i-1,j), f(i-1,j-w_i)+v_i)$$

改为滚动数组，从后向前遍历：

$$f(j) = (f(j), f(j-w_i)+v_i)$$

完全背包

每个物品可以选无限次个。$f(i,j)$ 表示选前 i 个物品，背包最大体积为 j 的情况下储存最大价值

$$f(i,j) = \max_{k=0}^{k\to+\infty}(f(i-1,j-kw_i)+kv_i)$$

优化至二维：

$$f(i,j) = \max(f(i-1,j), f(i,j-w_i)+v_i)$$

改为滚动数组，从前向后遍历：

$$f(j) = \max(f(j), f(j-w_i)+v_i)$$

多重背包

最多选 m 个物品的完全背包。$f(i,j)$ 表示选前 i 个物品，背包最大体积为 j 的情况下储存最大价值

$$f(i,j)=\max_{k=0}^{k\to m}(f(i-1,j-kw_i)+kv_i)$$

使用二进制分组优化可降低至 $O(nV\log m)$

分组背包

有 N 组物品，第 i 组物品有 n 个物品，每个物品有重量 w_i^k 和价值 v_i^k ，背包总容量 V

$$f(i,j)=\max\Big(f(i-1,j),\max_{k=1}^{k\to n}\big(f\big(i-1,j-w_i^k\big)+v_i^k\big)\Big)$$

3.2. 序列问题

最长上升子序列（LIS）

```
int n, res=0, a[N], dp[N];
int main() {
  cin >> n;
  for(int i = 0; i < n; i++)
    cin >> a[i];
  for(int i = 0; i < N; i++)
    dp[i] = 1;
  for(int i = 1; i < n; i++)
    for(int j = 0; j < i; j++)
      if(a[j] < a[i])
        dp[i] = max(dp[i], dp[j] + 1);
  for(int i = 0; i < n; i++)
    res = max(res, dp[i]);
  cout << res << endl;
}
```

最长公共子序列（LCS）

两个排列的 LCS 就是 LIS

```
const int N = 1e5 + 10;
// amp 表示 a 序列的值在 a 序列中的位置
int n, amp[N], b[N], f[N];
int main() {
  memset(f, 0x3f, sizeof(f));
  int x;
  cin >> n;
  for (int i = 1; i ≤ n; i++) cin >> x, amp[x] = i;
  for (int i = 1; i ≤ n; i++) cin >> b[i];
  int res = 0, t;
  f[0] = 0;
  for (int i = 1; i ≤ n; i++) {
    if (amp[b[i]] > f[res]) {
      f[++res] = amp[b[i]];
    } else {
      t = lower_bound(f + 1, f + res + 1, amp[b[i]]) - f;
      f[t] = min(amp[b[i]], f[t]);
    }
  }
  cout << res;
}
```

最长公共上升子序列（LICS）

```
int n;
int a[N], b[N];
int f[N][N];

int main() {
  cin >> n;
  for (int i = 1; i ≤ n; i++) cin >> a[i];
  for (int i = 1; i ≤ n; i++) cin >> b[i];

  for (int i = 1; i ≤ n; i++) {
    int maxv = 1;
    for (int j = 1; j ≤ n; j++) {
      f[i][j] = f[i - 1][j];
      if (a[i] == b[j]) f[i][j] = max(f[i][j], maxv);
      if (a[i] > b[j]) maxv = max(maxv, f[i - 1][j] + 1);
    }
  }
  int res = 0;
  for (int i = 1; i ≤ n; i++) res = max(res, f[n][i]);
  cout << res << endl;
}
```

4. 字符串

4.1. KMP

伪代码

- 输入主串 s，模式串 p，改为下标从 1 开始
- 计算 next 数组。i 是模式串的指针，每次循环计算 next[i]的值，j 可以理解为当前已经匹配的字符数量
  - 每当失配时，指针就跳转到 next[j]的位置，下标从 1 开始，略过 next[j]个字符，所以直接匹配了 j 个字符
  - 匹配成功，则匹配成功的字符数量+1
  - 记录匹配成功的字符数量
- KMP 算法。i 是主串的指针，j+1 是模式串的指针，j 也可以理解为当前已经匹配的字符数量
  - 每当失配时，j+1 就跳转到 next[j]+1 的位置继续匹配，也相当于略过 next[j]个字符
  - 匹配成功，则匹配成功的字符数量+1，模式串指针后移
  - 如果匹配数量等于模式串长度，说明匹配成功
    - 匹配成功后可以进行一些操作，例如输出匹配成功的位置
    - i-lenp+1 是匹配成功开始的位置，i 是匹配成功结束的位置

```
string input, s, p;
cin >> input; s = '#' + input;
cin >> input; p = '#' + input;
int lenp = p.size() - 1, lens = s.size() - 1;
vector<int> next;     // next数组的意义是，当模式串失配时，略过多少个字符
next.resize(lenp + 1);
for (int i = 2, j = 0; i ≤ lenp; i++) {
  while (j && p[i] ≠ p[j + 1])
    j = next[j];
  if (p[i] == p[j + 1])
    j++;
  next[i] = j;
}
for (int i = 1, j = 0; i ≤ lens; i++) {
  while (j && s[i] ≠ p[j + 1])
```

```
    j = next[j];
  if (s[i] == p[j + 1])
    j++;
  if (j == lenp) {
    j = next[j];
    // 匹配成功后的操作
    cout << i - lenp + 1 << endl;
  }
}
```

4.2. 字典树 Trie

```
int son[N][26], cnt[N], idx;
// 0号点既是根节点，又是空节点
// son[][]存储树中每个节点的子节点，cnt[]存储以每个节点结尾的单词数量
// 插入一个字符串
void insert(char *str) {
  in p = 0;
  for (int i = 0; str[i]; i++) {
    int u = str[i] - 'a';
    if (!son[p][u])
      son[p][u] = ++idx;
    p = son[p][u];
  }
  cnt[p]++;
}
// 查询字符串出现的次数
int query(char *str) {
  int p = 0;
  for (int i = 0; str[i]; i++) {
    int u = str[i] - 'a';
    if (!son[p][u])
      return 0;
    p = son[p][u];
  }
  return cnt[p];
}
```

4.3. AC 自动机

insert 所有模式串，调用 build 然后 find 文本串，返回匹配数量

```
struct Trie { // Trie树
  int son[26], cnt, fail;
} tr[N];
int tot = 0;
// 插入模式串
inline void insert(const string &s) {
  int u = 0;
  for (int i = 0; s[i]; i++) {
    int v = s[i] - 'a';
    if (!tr[u].son[v])
      tr[u].son[v] = ++tot;
    u = tr[u].son[v];
  }
  tr[u].cnt++;
}
```

```
void build() {  // 构建fail边
  queue<int> q;
  for (int v = 0; v < 26; v++) {
    int c = tr[0].son[v];
    if (c) {
      tr[c].fail = 0;
      q.push(c);
    }
  }
  while (!q.empty()) {
    int u = q.front();
    q.pop();
    int f = tr[u].fail;
    for (int v = 0; v < 26; v++) {
      int c = tr[u].son[v];
      if (c) {
        tr[c].fail = tr[f].son[v];
        q.push(c);
      } else
        tr[u].son[v] = tr[f].son[v];
    }
  }
}
// 计算模式串在文本串中出现的次数
inline int find(const string &s) {
  int u = 0, sum = 0;
  for (int i = 0; s[i]; i++) {
    int v = s[i] - 'a';
    int c = tr[u].son[v];
    while (c && tr[c].cnt != -1) {
      sum += tr[c].cnt;
      tr[c].cnt = -1;
      c = tr[c].fail;
    }
    u = tr[u].son[v];
  }
  return sum;
}
```

4.4. Manacher

求字符串的最长回文子串长度

```
char a[N], b[N];
int p[N * 2], n;
// a[]为原串，b[]为插入'#'后的新串
// p[i]表示以b[i]为中心的回文串半径
void init() {
  n = strlen(a);
  int k = 0;
  b[k++] = '$', b[k++] = '#';
  for (int i = 0; i < n; i++)
    b[k++] = a[i], b[k++] = '#';
  b[k++] = '^';
  n = k;
}
```

```
void manacher() {
  int mr = 0, mid;
  for (int i = 1; i < n; i++) {
    if (i < mr)
      p[i] = min(p[mid * 2 - i], mr - i);
    else
      p[i] = 1;
    while (b[i - p[i]] == b[i + p[i]])
      p[i]++;
    if (i + p[i] > mr) {
      mr = i + p[i];
      mid = i;
    }
  }
}
// 最长回文串长度
*max_element(p, p + n) - 1
```

5. 数学

5.1. 质因数

5.1.1. 质数筛

埃氏筛

```
// 找到0-n的所有质数
vector<bool> isPrime(n + 1, 1);
isPrime[1] = 0, isPrime[0] = 0;
int e = sqrt(n);
for (int i = 2; i ≤ e; i++)
  if (isPrime[i] == 1)
    for (int j = i + i; j ≤ n; j += i)
      isPrime[j] = 0;
```

线性筛 / 欧拉筛

```
vector<int> isPrime(n + 1, 1);
vector<int> primes;
isPrime[0] = isPrime[1] = 0;
for (int i = 2; i ≤ n; i++) {
  if (isPrime[i])
    primes.push_back(i);
  for (int k : primes) {
    if (i * k > n)
      break;
    isPrime[i * k] = 0;
    if (i % k == 0)
      break;
  }
}
```

5.1.2. LCM & GCD

原理：$\gcd(a, b) = \gcd(b, a \bmod b)$ 特殊地 $\gcd(a, 0) = a$

```
int gcd(int a, int b) {
  return b ? gcd(b, a % b) : a;
}
```

```
int lcm(int a,int b){
  return (a / gcd(a, b)) * b;   // 先除再乘，避免超过数据范围
}
int gcd(int a, int b) {  // 非递归
  while (b) { int r = a % b; a = b; b = r; }
  return a;
}
```

5.1.3. 质因数分解和因数问题

```
void divide(int x) {
  for (int i = 2; i ≤ x / i; i++)
    if (x % i == 0) {
      int s = 0;
      while (x % i == 0)
        x /= i, s++;
      cout << i << ' ' << s << endl;
    }
  if (x > 1)
    cout << x << ' ' << 1 << endl;
  cout << endl;
}
```

如果一个数质因数分解为:

$$n = p_1^{a_1} \times p_2^{a_2} \times p_3^{a_3} \times ..... \times p_k^{a_k}$$

那么它的因数个数为:

$$(a_1+1)(a_2+1)(a_3+1).....(a_k+1)$$

因数和为:

$$\sum_{j=0}^{a_1} p_1^j \times \sum_{j=0}^{a_2} p_2^j \times \sum_{j=0}^{a_3} p_3^j \times ..... \times \sum_{j=0}^{a_k} p_k^j$$

5.2. 欧拉函数

5.2.1. 欧拉定理

如果一个数质因数分解为:

$$n = p_1^{a_1} \times p_2^{a_2} \times p_3^{a_3} \times ..... \times p_k^{a_k}$$

欧拉函数的定义为:

$$\varphi(n) = n\left(1-\frac{1}{p_1}\right)\left(1-\frac{1}{p_2}\right)...\left(1-\frac{1}{p_k}\right)$$

欧拉函数和质因数的幂次无关

```
int phi(int x)
{
  int res = x;
  for (int i = 2; i ≤ x / i; i ++ )
    if (x % i == 0)
    {
      res = res / i * (i - 1);
      while (x % i == 0) x /= i;
    }
```

```
  if (x > 1) res = res / x * (x - 1);

  return res;
}
```

若有质数 p 则:

$$\varphi(pn) = p\varphi(n)\left(1 - \frac{1}{p}\right) = (p-1)\varphi(n)$$

欧拉函数的乘法性质:

$$\varphi(a \cdot b) = \varphi(a) \cdot \varphi(b)$$

欧拉函数的质数性质:

$$\varphi(p) = p - 1$$

欧拉函数的幂性质:

$$\varphi(pk) = p^k - p^{k-1} = p^k\left(1 - \frac{1}{p}\right)$$

使用筛法的欧拉函数

```
int primes[N], cnt; // primes[]存储所有素数
int euler[N];       // 存储每个数的欧拉函数
bool st[N];         // st[x]存储x是否被筛掉

void get_eulers(int n) {
  euler[1] = 1;
  for (int i = 2; i ≤ n; i++) {
    if (!st[i]) {
      primes[cnt++] = i;
      euler[i] = i - 1;
    }
    for (int j = 0; primes[j] ≤ n / i; j++) {
      int t = primes[j] * i;
      st[t] = true;
      if (i % primes[j] == 0) {
        euler[t] = euler[i] * primes[j];
        break;
      }
      euler[t] = euler[i] * (primes[j] - 1);
    }
  }
}
```

5.2.2. 费马小定理

欧拉定理

- 若 $\gcd(a, n) = 1$ 则 $a^{\varphi(n)} \equiv 1 (\bmod n)$

例如 5 和 6 互质，$\varphi(6) = 2$，$5^2 \bmod 6 = 1$

费马小定理

- 若 p 为质数且 $\gcd(a, p) = 1$ 则 $a^{p-1} \equiv 1 (\bmod p)$
- 对于任意整数 a，$a^p \equiv 1 (\bmod p)$

5.3. 快速幂和乘法逆元

```
// 快速幂: 求 m 的 k 次方 mod p
int qmi(int a, int k, int p) {
  int res = 1;
  while (k) {
    if (k & 1) res = (LL)res * a % p;
    a = (LL)a * a % p;
    k >>= 1;
  }
  return res;
}
```

要求 $a \div b \bmod p$，即求 $a \times b^{-1} (\bmod p)$

$$b^{-1} \equiv b^{p-2} (\bmod p)$$

前提条件：p 为质数，b, p 互质，否则需要用扩展欧几里得算法求逆元

```
// 扩展欧几里得算法求逆元
int mmi(int b, int p) {
  int x, y;
  int g = std::__gcd(b, p); // 可选的合法性判断（快速）
  if (g != 1) return -1;    // b 和 p 不互质，无逆元
  exgcd(b, p, x, y);        // 解 bx + py = 1
  x = (x % p + p) % p;      // 保证 x 为正，且 x < p
  return x;
}
// exgcd 函数如下
```

5.4. 扩展欧几里得算法

```
// 求 ax + by = gcd(a, b) 的整数解
int exgcd(int a, int b, int &x, int &y) {
  if (!b) {
    x = 1; y = 0;
    return a;
  }
  int d = exgcd(b, a % b, y, x);
  y -= (a/b) * x;
  return d;
}
// 解线性同余方程 ax ≡ b (mod m)
// 返回最小非负整数解 x（模 m/d），如果无解返回 -1
int solve_mod_linear(int a, int b, int m) {
  int x, y;
  int d = std::gcd(a, m);       // 等价于你用 exgcd 求 d
  if (b % d != 0) return -1;    // 无解

  exgcd(a, m, x, y);            // 解 ax + my = d
  x = x * (b / d);              // 得到 ax = b 的特解
  int mod = m / d;
  x = (x % mod + mod) % mod;    // 化成最小非负解
  return x;
}
```

5.5. 中国剩余定理

给定一组两两互质的数：$m_1, m_2, m_3 ... m_k$，有一组数 $a_1, a_2, a_3 ... a_k$ 满足 $x \equiv a_i (\bmod m_i), i \in [1, k]$ 这个方程组的公式解为：

$$x = \sum_{i=1}^{k} (a_i \cdot M_i \cdot M_i^{-1})$$

```
// 中国剩余定理，求解 x ≡ a[i] (mod m[i])
ll CRT(ll a[], ll m[], int n) {
  ll PM = 1, Mi, MiR, tmp, res = 0;
  for (int i = 1; i ≤ n; ++i)
    PM *= m[i];
  for (int i = 1; i ≤ n; ++i) {
    Mi = PM / m[i];
    MiR = mmi(Mi, m[i]);   // 扩展欧几里得求逆元
    tmp = a[i] * Mi * MiR % PM;
    res = (res + tmp) % PM;
  }
  return res;
}
```

5.6. 线性代数

5.6.1. 高斯消元

求解线性方程组 $A_x = b$，其中 A 是 $n \times n$ 的系数矩阵，b 是 $n \times 1$ 的常数向量，增广矩阵为 $[A|b]$。

```
// a[N][N]是增广矩阵
int gauss() {
  int c, r;
  for (c = 0, r = 0; c < n; c ++ ) {
    int t = r;
    for (int i = r; i < n; i ++ )   // 找到绝对值最大的行
      if (fabs(a[i][c]) > fabs(a[t][c]))
        t = i;

    if (fabs(a[t][c]) < eps) continue;

    // 将绝对值最大的行换到最顶端
    for (int i = c; i ≤ n; i ++ ) swap(a[t][i], a[r][i]);
    // 将当前行的首位变成1
    for (int i = n; i ≥ c; i -- ) a[r][i] /= a[r][c];
    // 用当前行将下面所有的列消成0
    for (int i = r + 1; i < n; i ++ )
      if (fabs(a[i][c]) > eps)
        for (int j = n; j ≥ c; j -- )
          a[i][j] -= a[r][j] * a[i][c];
    r ++ ;
  }
  if (r < n) {
    for (int i = r; i < n; i ++ )
      if (fabs(a[i][n]) > eps)
        return 2; // 无解
    return 1; // 有无穷多组解
  }
  for (int i = n - 1; i ≥ 0; i -- )
```

```
    for (int j = i + 1; j < n; j ++ )
      a[i][n] -= a[i][j] * a[j][n];
  return 0; // 有唯一解
}
```

5.7. 组合数学

$$A_a^b = a(a-1)(a-2)...(a-b+1) = \frac{a!}{(a-b)!}$$

$$C_a^b = \frac{a(a-1)(a-2)c...(a-b+1)}{1\times 2\times 3...\times b} = \frac{a!}{b!(a-b)!} = \frac{A_a^b}{b!}$$

组合数算法:

- 递推法：$n \le 10^5, 1 \le b \le a \le 2000, O(n^2)$
- 预处理：$n \le 10^5, 1 \le b \le a \le 10^5, O(n\log n)$
- Lucas：$n \le 20, 1 \le b \le a \le 10^{18}, 1 \le p \le 10^5, O(p\log n\log p)$
- 分解质因数：$1 \le b \le a \le 5000$, 不取模

5.7.1. 递推法

$$C_a^b = C_{a-1}^b + C_{a-1}^{b-1}$$

```
// c[a][b] 表示从 a 个中选 b 个的方案数
int c[N][N]={0};
for (int i = 0; i < N; i ++ )
    for (int j = 0; j ≤ i; j ++ )
        if (!j) c[i][j] = 1;
        else c[i][j] = (c[i - 1][j] + c[i - 1][j - 1]) % mod;
```

5.7.2. 预处理法

```
// 预处理阶乘的余数和阶乘逆元的余数
fact[0] = infact[0] = 1;
for (int i = 1; i < N; i ++ )
{
    fact[i] = (LL)fact[i - 1] * i % mod;
    infact[i] = (LL)infact[i - 1] * qmi(i, mod - 2, mod) % mod;
}
int C(int n, int m) {
    if (m < 0 || m > n) return 0;
    return (LL)fact[n] * infact[m] % mod * infact[n - m] % mod;
}
```

5.7.3. Lucas 定理

```
int qmi(int a, int k, int p); // 如上
// 通过定理求组合数C(a, b)
int C(int a, int b, int p) {
  if (a < b) return 0;
  LL x = 1, y = 1;  // x是分子, y是分母
  for (int i = a, j = 1; j ≤ b; i--, j++ )
    x = (LL)x * i % p,
    y = (LL) y * j % p;

  return x * (LL)qmi(y, p - 2, p) % p;
}
int lucas(LL a, LL b, int p) {
  if (a < p && b < p) return C(a, b, p);
```

```
  return (LL)C(a % p, b % p, p) * lucas(a / p, b / p, p) % p;
}
```

5.7.4. 分解质因数

```
int primes[N], cnt;     // 存储所有质数
int sum[N];     // 存储每个质数的次数
bool st[N];     // 存储每个数是否已被筛掉
// 线性筛
void get_primes(int n) {
  for (int i = 2; i ≤ n; i ++ ) {
    if (!st[i]) primes[cnt ++ ] = i;
    for (int j = 0; primes[j] ≤ n / i; j ++ ) {
      st[primes[j] * i] = true;
      if (i % primes[j] == 0) break;
    }
  }
}
// 求n！中的次数
int get(int n, int p) {
  int res = 0;
  while (n) {
    res += n / p;
    n /= p;
  }
  return res;
}
// 高精度乘低精度
vector<int> mul(vector<int> a, int b) {
  vector<int> c;
  int t = 0;
  for (int i = 0; i < a.size(); i ++ ) {
    t += a[i] * b;
    c.push_back(t % 10);
    t /= 10;
  }
  while (t) {
    c.push_back(t % 10);
    t /= 10;
  }
  return c;
}

get_primes(a);  // 预处理范围内的所有质数
// 求每个质因数的次数
for (int i = 0; i < cnt; i ++ ) {
  int p = primes[i];
  sum[i] = get(a, p) - get(b, p) - get(a - b, p);
}
vector<int> res;
res.push_back(1);
for (int i = 0; i < cnt; i ++ )     // 用高精度乘法将所有质因子相乘
  for (int j = 0; j < sum[i]; j ++ )
    res = mul(res, primes[i]);
```

5.8. 卡特兰数

H_0到H_9的卡特兰数为:

$$1, 1, 2, 5, 14, 42, 132, 429, 1430, 4862$$

递推公式:

$$H_n = \frac{\left\{H_{\{n-1\}}(4n-2)\right\}}{\{n+1\}}$$

$$H_n = C_{\{2n\}}^{\{n\}} - C_{\{2n\}}^{\{n-1\}}$$

$$H_n = \sum_{\{i=1\}}^{n} H_{\{i-1\}} H_{\{n-i\}}$$

封闭形式:

$$H_n = \frac{\left\{C_{\{2n\}}^{n}\right\}}{\{n+1\}}$$

上述公式中 $n \geq, n \in N_+, H_0 = 1, H_1 = 1$

应用于以下问题:

- 有 $2n$ 个人排成一行进入剧场。入场费 5 元。其中只有 n 个人有一张 5 元钞票，另外 n 人只有 10 元钞票，剧院无其它钞票，问有多少种方法使得只要有 10 元的人买票，售票处就有 5 元的钞票找零?
- 有一个大小为 $n \times n$ 的方格图左下角为 $(0, 0)$ 右上角为 (n, n)，从左下角开始每次都只能向右或者向上走一单位，不走到对角线 $y = x$ 上方（但可以触碰）的情况下到达右上角有多少可能的路径?
- 在圆上选择 的$2n$ 个点，将这些点成对连接起来使得所得到的 n 条线段不相交的方法数?
- 对角线不相交的情况下，将一个凸多边形区域分成三角形区域的方法数?
- 一个栈（无穷大）的进栈序列为 $1, 2, 3, ..., n$ 有多少个不同的出栈序列?
- n 个结点可构造多少个不同的二叉树?
- 由 n 个 $+1$ 和 n 个 -1 组成的 $2n$ 个数 $a_1, a_2, ..., a_{\{2n\}}$，其部分和满足 $a_1 + a_2 + ... + a_k \geq 0 (k = 1, 2, 3, ..., 2n)$，有多少个满足条件的数列?

5.9. 简单博弈论

5.9.1. Nim 游戏

有 n 堆重量为 $a_1, a_2, a_3, ..., a_n$ 的石子，两名玩家轮流从一堆石子中取出任意个石子，最后没石子可取的人判输

如果 $a_1 \oplus a_2 \oplus a_3 \oplus ... \oplus a_n$ 为 0，则先手必败，如果不为零，则先手必胜

6. 数据结构

6.1. 并查集

```
int p[N]; //存储每个点的祖宗节点

// 返回x的祖宗节点
int find(int x)
{
  if (p[x] != x) p[x] = find(p[x]); // 路径压缩
  return p[x];
}

// 初始化，假定节点编号是1~n
```

```
for (int i = 1; i ≤ n; i ++ ) p[i] = i;

// 合并a和b所在的两个集合:
p[find(a)] = find(b);
```

6.2. ST 表

如果需要区间 GCD 就把两个 max 改成 gcd

```
int n, st[N][lgN + 1], lg[N + 1];
void build() {
  // 预处理所有的 ⌊log(n)⌋
  lg[1] = 0, lg[2] = 1;
  for (int i = 3; i < N; i++)
    lg[i] = lg[i / 2] + 1;
  // 构建 ST 表
  for (int j = 1; j ≤ lgN; j++)
    for (int i = 1; i + (1 << j) - 1 ≤ n; i++)
      st[i][j] = max(st[i][j - 1], st[i + (1 << (j - 1))][j - 1]);
}
// 查询区间[l, r]的最大值
inline int query(int l, int r) {
  int k = lg[r - l + 1];
  return max(st[l][k], st[r - (1 << k) + 1][k]);
}
// 读入 n 个数
for (int i = 1; i ≤ n; i++)
  cin >> st[i][0];
```

6.3. 堆

```
int h[N];  // 存储堆中的值, h[1]是堆顶, x的左儿子是2x, 右儿子是2x + 1
int ph[N]; // 存储第k个插入的点在堆中的位置
int hp[N]; // 存储堆中下标是i的点是第几个插入的
int hsize, idx = 0;
void heap_swap(int a, int b) {  // 交换两个点
  swap(ph[hp[a]], ph[hp[b]]);
  swap(hp[a], hp[b]);
  swap(h[a], h[b]);
}
void down(int u) {
  int t = u;
  if (u * 2 ≤ hsize && h[u * 2] < h[t])
    t = u * 2;
  if (u * 2 + 1 ≤ hsize && h[u * 2 + 1] < h[t])
    t = u * 2 + 1;
  if (u ≠ t) {
    heap_swap(u, t);
    down(t);
  }
}
void up(int u) {
  while (u / 2 && h[u] < h[u / 2]) {
    heap_swap(u, u / 2);
    u >>= 1;
  }
}
```

```
void build() {  // 原地建堆O(n)
  cin >> hsize;
  idx = hsize;
  for (int i = 1; i ≤ hsize; i++)
    cin >> h[i], ph[i] = i, hp[i] = i;
  for (int i = hsize / 2; i; i--)
    down(i);
}
void push(int x) {
  hsize++;
  idx++;
  ph[idx] = hsize;
  hp[hsize] = idx;
  h[hsize] = x;
  up(hsize);
}
void remove(int k) {  // 删除第 k 个插入的元素
  int pos = ph[k];
  heap_swap(pos, hsize); // 交换到堆尾
  hsize--; // 删除
  down(pos);up(pos);
}
void change(int k, int x) { // 修改第 k 个插入的元素为 x
  int pos = ph[k];
  h[pos] = x;      // 修改值
  down(pos);up(pos);
}
remove(hp[1]);    // 弹出堆顶，记得判断hsize是否为0
h[1];             // 堆顶，记得判断hsize是否为0
```

6.4. 树状数组

单点改区间查

```
int n, tr[N];     // 数据个数、树状数组
int a[N];
int lowbit(int x) { return x & -x; }
// 将x位置加上c
void update(int x, int c) {
  for (int i = x; i ≤ n; i += lowbit(i))
    tr[i] += c;
}
// 查询[1,x]的和
int query(int x) {
  int res = 0;
  for (int i = x; i; i -= lowbit(i))
    res += tr[i];
  return res;
}
// 快速初始化(区间和)
int pre[N];  // 前缀和数组
for (int i = 1; i ≤ n; i++)
  cin >> a[i], pre[i] = pre[i - 1] + a[i];
for (int i = 1; i ≤ n; i++)
  tr[i] = pre[i] - pre[i - lowbit(i)];
// 初始化(通用)
```

```
for (int i = 1; i ≤ n; i++)
cin >> a[i], update(i, a[i]);

// 维护区间异或和: 只需修改两个函数的 += 为 ^=
// 维护前缀最大值: 只需修改两函数 += 为 max 函数
// 二维树状数组: 将tr改为二维, 两函数增加一层循环
```

区间改区间查

```
int t1[N], t2[N], n;
int lowbit(int x) { return x & (-x); }
void add(int k, int v) {
  int v1 = k * v;
  while (k ≤ n) {
    t1[k] += v, t2[k] += v1;
    // 注意不能写成 t2[k] += k * v, 因为 k 的值已经不是原数组的下标了
    k += lowbit(k);
  }
}
int getsum(int *t, int k) {
  int ret = 0;
  while (k) {
    ret += t[k];
    k -= lowbit(k);
  }
  return ret;
}
void add1(int l, int r, int v) {
  add(l, v), add(r + 1, -v);  // 将区间加差分为两个前缀加
}
ll getsum1(int l, int r) {
  return (r + 1ll) * getsum(t1, r) - 1ll * l * getsum(t1, l - 1) -
         (getsum(t2, r) - getsum(t2, l - 1));
}
```

6.5. 线段树

6.5.1. 朴素线段树

支持区间查询单点修改

区间最大值

```
int n;
struct Node {
  int l, r, val;
} tr[N * 4];
void pushup(int u) { tr[u].val = max(tr[u << 1].val, tr[u << 1 | 1].val); }
// build(1, 1, n) 建树
void build(int u, int l, int r) {
  // 1、记录区间信息
  tr[u].l = l;
  tr[u].r = r;
  tr[u].val = 0;
  // 2、如果区间仅有一个数返回
  if (l == r) {
    tr[u].val = 0;
    return;
```

```
  }
  // 3、递归建树
  int mid = (l + r) >> 1;
  build(u << 1, l, mid);
  build(u << 1 | 1, mid + 1, r);
  // 4、pushup，由于建空树，所以不需要
}
// query(1, l, r) 查询[l,r]区间的最大值
int query(int u, int l, int r) {
  // 情况1:被包含
  if (l ≤ tr[u].l && tr[u].r ≤ r)
    return tr[u].val;
  // 情况2:有交集
  int mid = (tr[u].l + tr[u].r) >> 1;
  int tmp = INT_MIN;
  if (l ≤ mid)
    tmp = max(tmp, query(u << 1, l, r));
  if (mid < r)
    tmp = max(tmp, query(u << 1 | 1, l, r));
  return tmp;
}
//  modify(1, x, val) 将单点 x 修改为val
void modify(int u, int x, int val) {
  if (tr[u].l == x && tr[u].r == x) {
    tr[u].val = val;
    return;
  }
  int mid = (tr[u].l + tr[u].r) >> 1;
  if (x ≤ mid)
    modify(u << 1, x, val);
  else
    modify(u << 1 | 1, x, val);
  pushup(u);
}
```

区间最大子段和

```
int n, a[N];
struct Node {
  int l, r; // 区间端点
  int tmax, lmax, rmax, sum; // 子段和 前缀和 后缀和 区间和
} tr[N * 4];
inline void pushup(Node &u, Node &l, Node &r) {
  u.sum = l.sum + r.sum;
  u.tmax = max(l.rmax + r.lmax, max(l.tmax, r.tmax));
  u.lmax = max(l.lmax, l.sum + r.lmax);
  u.rmax = max(r.rmax, r.sum + l.rmax);
}
inline void pushup(int u) { pushup(tr[u], tr[u << 1], tr[u << 1 | 1]); }
// build(1, 1, n) 用 a[n] 建树
void build(int u, int l, int r) {
  tr[u].l = l;
  tr[u].r = r;
  if (l == r) {
    tr[u].lmax = tr[u].rmax = tr[u].tmax = tr[u].sum = a[l];
```

```
    return;
  }
  int mid = (l + r) >> 1;
  build(u << 1, l, mid);
  build(u << 1 | 1, mid + 1, r);
  pushup(u);
}
// query(1, x, y).tmax 查询区间[x,y]的最大子段和
Node query(int u, int l, int r) {
  if (l ≤ tr[u].l && tr[u].r ≤ r)
    return tr[u];
  int mid = (tr[u].l + tr[u].r) >> 1;
  if (r ≤ mid)
    return query(u << 1, l, r);
  else if (l > mid)
    return query(u << 1 | 1, l, r);
  else {
    Node left = query(u << 1, l, r);
    Node right = query(u << 1 | 1, l, r);
    Node res;
    pushup(res, left, right);
    return res;
  }
}
// modify(1, x, y) 将单点x修改为y
void modify(int u, int x, int v) {
  if (tr[u].l == x && tr[u].r == x) {
    tr[u].lmax = tr[u].rmax = tr[u].tmax = tr[u].sum = v;
    return;
  }
  int mid = (tr[u].l + tr[u].r) >> 1;
  if (x ≤ mid)
    modify(u << 1, x, v);
  else
    modify(u << 1 | 1, x, v);
  pushup(u);
}
```

区间最大公约数

```
ll n, m, a[N];
struct Node {
  int l, r;
  ll gcd, sum; // 区间和sum也要维护
} tr[4 * N];   // 此线段树维护的是差分
ll gcd(ll a, ll b) {
  return b ? gcd(b, a % b) : llabs(a); // 要考虑负数
}
void pushup(Node &u, Node &l, Node &r) {
  u.gcd = gcd(l.gcd, r.gcd);
  u.sum = l.sum + r.sum;
}
void pushup(int u) {
  pushup(tr[u], tr[u << 1], tr[u << 1 | 1]); // 所有的访问子节点都是<<
}
```

```
// build(1, 1, n) 用 a[n] 建树
void build(int u, int l, int r) {
  // 1、记录区间端点
  tr[u].l = l;
  tr[u].r = r;
  // 2、如果找到叶节点，输入信息并返回
  if (l == r) {
    tr[u].gcd = a[l] - a[l - 1];
    tr[u].sum = a[l] - a[l - 1];  // 差分
    return;
  }
  // 3、递归求子区间
  int mid = (l + r) >> 1; // 所有的求mid都是>>
  build(u << 1, l, mid);
  build(u << 1 | 1, mid + 1, r);
  // 4、子区间全部计算完成，pushup当前节点，给当前节点赋值
  pushup(u);
}
void modify(int u, int x, ll v) { // v是数值不是区间要ll
  // 1、如果找到叶节点就修改值并返回
  if (tr[u].l == x && tr[u].r == x) {
    tr[u].sum += v;
    tr[u].gcd = abs(tr[u].sum); // 处理负数
    return;                     // 找到叶节点记得立刻返回
  }
  // 2、如果没有找到递归找子区间
  int mid = (tr[u].l + tr[u].r) >> 1;
  if (x <= mid)
    modify(u << 1, x, v);
  else
    modify(u << 1 | 1, x, v);
  // 3、下方完成计算，pushup当前节点
  pushup(u);
}
Node query(int u, int l, int r) {
  // 情况1、区间完全被包含
  if (l <= tr[u].l && tr[u].r <= r) {
    return tr[u]; // 在区间内了就返回
  }
  // 情况2和3、区间在左右子区间内
  int mid = (tr[u].l + tr[u].r) >> 1;
  if (r <= mid)
    return query(u << 1, l, r);
  else if (l > mid)
    return query(u << 1 | 1, l, r);
  // 情况4、区间横跨左右子区间
  else {
    Node left = query(u << 1, l, r);
    Node right = query(u << 1 | 1, l, r);
    Node res;
    pushup(res, left, right);
    return res;
  }
}
```

```
// 查询区间[l,r]的gcd
abs(query(1, 1, l).sum); // l == r 特判
gcd(query(1, 1, l).sum, query(1, l + 1, r).gcd);

// 将区间[l,r]的值加上d
modify(1, l, d);
if (r + 1 ≤ n) modify(1, r + 1, -d);
```

6.5.2. 惰性线段树

支持区间查询区间修改

区间加

```
int n, a[N];
struct Node {
  int l, r;
  ll sum;
  ll add;
} tr[N * 4];
void pushup(int u) { tr[u].sum = tr[u << 1].sum + tr[u << 1 | 1].sum; }
void pushdown(int p) {
  Node &u = tr[p], &l = tr[p << 1], &r = tr[p << 1 | 1];
  if (u.add != 0) {
    l.sum += (l.r - l.l + 1) * u.add;
    r.sum += (r.r - r.l + 1) * u.add;
    l.add += u.add;
    r.add += u.add;
    u.add = 0;
  }
}
// build(1, 1, n) 用a[n]建树
void build(int u, int l, int r) {
  tr[u].l = l;
  tr[u].r = r;
  if (l == r) {
    tr[u].sum = a[l];
    return;
  }
  int mid = (l + r) >> 1;
  build(u << 1, l, mid);
  build(u << 1 | 1, mid + 1, r);
  pushup(u);
}
// modify(1, x, y, k) 给区间[x,y]加上k
void modify(int p, int l, int r, ll d) {
  Node &u = tr[p];
  if (l ≤ u.l && u.r ≤ r) {
    u.sum += (u.r - u.l + 1) * d;
    u.add += d;
    return;
  }
  pushdown(p);
  int mid = (u.l + u.r) >> 1;
  if (l ≤ mid)
    modify(p << 1, l, r, d);
  if (r > mid)
```

```
    modify(p << 1 | 1, l, r, d);
  pushup(p);
}
// query(1, x, y) 求区间[x,y]的和
ll query(int p, int l, int r) {
  Node &u = tr[p];
  if (l ≤ u.l && u.r ≤ r)
    return u.sum;
  pushdown(p);
  int mid = (u.l + u.r) >> 1;
  ll res = 0;
  if (l ≤ mid)
    res += query(p << 1, l, r);
  if (r > mid)
    res += query(p << 1 | 1, l, r);
  return res;
}
```

区间加区间乘

```
int n, MOD, a[N];
struct Node {
  int l, r, sum, add, mul;
} tr[N * 4];
void pushup(int u) {
  // 任何有设计加法乘法都先强制转换为ll
  tr[u].sum = ((ll)tr[u << 1].sum + tr[u << 1 | 1].sum) % MOD;
}
// 关键在于两个互相影响的懒标记如何维护: calc和pushdown函数
void calc(Node &t, int add, int mul) {
  t.sum = ((ll)t.sum * mul + (ll)(t.r - t.l + 1) * add) % MOD;
  t.mul = (ll)t.mul * mul % MOD;
  t.add = ((ll)t.add * mul + add) % MOD;
}
void pushdown(int u) {
  calc(tr[u << 1], tr[u].add, tr[u].mul);
  calc(tr[u << 1 | 1], tr[u].add, tr[u].mul);
  tr[u].add = 0, tr[u].mul = 1;
}
// build(1, 1, n) 用 a[n] 建树
void build(int p, int l, int r) {
  Node &u = tr[p];
  u.l = l, u.r = r;
  u.add = 0, u.mul = 1;
  if (l == r) {
    u.sum = a[l];
    return;
  }
  int mid = (l + r) >> 1;
  build(p << 1, l, mid);
  build(p << 1 | 1, mid + 1, r);
  pushup(p);
}
// modify(1, x, y, a, d) 将区间先乘以 d 再加上 a
void modify(int p, int l, int r, int add, int mul) {
  Node &u = tr[p];
```

```
  if (l ≤ u.l && u.r ≤ r) {
    calc(u, add, mul);
    return;
  }
  pushdown(p);
  int mid = (u.l + u.r) >> 1; // 这里不是修改的区间的中点，不是(l+r)>>1
  if (l ≤ mid)
    modify(p << 1, l, r, add, mul);
  if (r > mid)
    modify(p << 1 | 1, l, r, add, mul);
  pushup(p);
}
// query(1, x, y) 查询区间 [x,y] 的和
ll query(int p, int l, int r) {
  Node &u = tr[p];
  if (l ≤ u.l && u.r ≤ r)
    return u.sum;
  pushdown(p);
  int mid = (u.l + u.r) >> 1;
  ll res = 0; // 这里的mid同理
  if (l ≤ mid)
    res = query(p << 1, l, r);
  if (r > mid)
    res += query(p << 1 | 1, l, r);
  res %= MOD;
  return res;
}
```

6.6. 平衡树

6.6.1. Treap

```
int n;
struct Node {
  int l, r;
  int k;
  int val; // 堆中的编号
  int cnt, size;
} tr[N];
int root, idx;
void pushup(int u) {
  tr[u].size =
      tr[tr[u].l].size + tr[tr[u].r].size + tr[u].cnt; // 上传节点信息，更新size
}
int new_node(int k) {
  tr[++idx].k = k;
  tr[idx].val = rand(); // 尽量随机，随手给个就行
  tr[idx].cnt = 1;
  tr[idx].size = 1;
  return idx;
}
void zig(int &u) // 左旋
{
  int q = tr[u].l;
  tr[u].l = tr[q].r;
  tr[q].r = u;
```

```
  u = q;
  pushup(tr[u].r);
  pushup(u); // 最后一定要记得上传，不然完了
}
void zag(int &u) { // 右旋
  int q = tr[u].r;
  tr[u].r = tr[q].l;
  tr[q].l = u;
  u = q;
  pushup(tr[u].l);
  pushup(u);
}
void build() // 建树操作，为了正确性增加两个哨兵，防止越界
{
  new_node(-INF), new_node(INF);
  root = 1, tr[1].r = 2; // 初始化一下
  pushup(root);          // 上传信息
  if (tr[1].val < tr[2].val)
    zag(root); // 不平衡了就旋转
}
void insert(int &u, int k) {
  if (u == 0)
    u = new_node(k); // 如果走到空了，就新建
  else {
    if (tr[u].k == k) // 如果找到了相同的节点，就cnt++
    {
      tr[u].cnt++;
    } else {
      if (tr[u].k > k) // 否则看看是在左边还是在右边
      {
        insert(tr[u].l, k);
        if (tr[tr[u].l].val > tr[u].val)
          zig(u); // 不平衡立马调整
      } else {
        insert(tr[u].r, k);
        if (tr[tr[u].r].val > tr[u].val)
          zag(u);
      }
    }
  }
  pushup(u); // 最后上传一下
}
void del(int &u, int k) // 删除操作
{
  if (u == 0)
    return;         // 如果没了说明节点不存在，就不管了。
  if (tr[u].k == k) // 如果找到了这个点
  {
    if (tr[u].cnt > 1)
      tr[u].cnt--; // 大于一好说，直接cnt --
    else           // 不大于一
    {
      if (tr[u].l || tr[u].r) // 先看看是不是叶节点
      {
```

```
        if (!tr[u].r || tr[tr[u].l].val) {
          zig(u);
          del(tr[u].r, k); // 记得维护平衡
        } else {
          zag(u);
          del(tr[u].l, k);
        }
      } else
        u = 0; // 是的话不用考虑平衡问题，直接删就是了
    }
  } else if (tr[u].k > k)
    del(tr[u].l, k); // 如果没有找到就判断一下在左右两边的哪一边
  else
    del(tr[u].r, k); // 找一下
  pushup(u);         // 上传更改
}
int get_rank(int u, int k) {
  if (u == 0)
    return 1; // 是 0 随便返回就行
  if (tr[u].k == k)
    return tr[tr[u].l].size + 1; // 相等了那排名应该就是左边的数量加上自己
  if (tr[u].k > k)
    return get_rank(tr[u].l, k);                              // 大了找左边
  return tr[tr[u].l].size + tr[u].cnt + get_rank(tr[u].r, k); // 找右边
}
int get_key(int u, int rank) {
  if (u == 0)
    return INF;
  if (tr[tr[u].l].size ≥ rank)
    return get_key(tr[u].l, rank); // 找左边
  if (tr[tr[u].l].size + tr[u].cnt ≥ rank)
    return tr[u].k; // 如果满足条件就直接return
  return get_key(tr[u].r, rank - tr[tr[u].l].size - tr[u].cnt); // 不然就找右边
}
int get_pr(int u, int k) // 前驱
{
  if (u == 0)
    return -INF;
  if (tr[u].k ≥ k)
    return get_pr(tr[u].l, k); // 找左边
  return max(get_pr(tr[u].r, k), tr[u].k); // 可能是右边可能是这个数，所以用个max
}
int get_ne(int u, int k) // 后继
{
  if (u == 0)
    return INF; // 后继的写法和前驱相反，大家可以注意一下
  if (tr[u].k ≤ k)
    return get_ne(tr[u].r, k);
  return min(get_ne(tr[u].l, k), tr[u].k);
}
int main() {
  build(); // 建树，要是忘了就凉了
  cin >> n;
  while (n--) {
```

```
    int op, x;
    cin >> op >> x;
    if (op == 1)          // 插入数值 x
      insert(root, x);
    else if (op == 2)     // 删除数值 x（若有多个相同的数，应只删除一个）
      del(root, x);
    else if (op == 3)     // 查询数值 x 的排名(若有多个相同的数，应输出最小的排名)
      cout << get_rank(root, x) - 1 << endl;
    else if (op == 4)     // 查询排名为 x 的数值
      cout << get_key(root, x + 1) << endl;
    else if (op == 5)     // 求数值 x 的前驱(前驱定义为小于 x 的最大的数)
      cout << get_pr(root, x) << endl;
    else if (op == 6)     // 求数值 x 的后继(后继定义为大于 x 的最小的数)。
      cout << get_ne(root, x) << endl;
  }
  return 0;
}
```

7. 图论

7.1. 基础

链式前向星

```
// 如果是无向图 M 要乘 2
int h[N], e[M], ne[M], idx = 0;
// 添加一条边 a→b
void add(int a, int b)
{
  e[idx] = b;
  ne[idx] = h[a];
  h[a] = idx++;
}
// 记得初始化
memset(h, -1, sizeof h);
```

拓扑排序

```
queue<int> q; // 所有0入度点
vector<int> topo; // 拓扑序答案
while(!q.empty()){
  int tmp = q.front();
  topo.push_back(tmp);
  q.pop();
  while(枚举点tmp所有的出边){
    删除该边
    d[j]--; // 指向的点入度减一
    if(d[j] == 0) q.push_back(j);
  }
}
```

7.2. 最短路

- 稠密图：朴素 Dijkstra $O(n^2)$

- 稀疏图：堆优化 Dijkstra $O(m \log n)$
- 负边权：Bellman-Ford $O(nm)$
- 负边权常用：SPFA $O(m)$
- 多源汇：Floyd $O(n^3)$

7.2.1. 朴素 Dijkstra

临接矩阵模板

```
int g[N][N]; // 存储每条边
int dist[N]; // 存储1号点到每个点的最短距离
bool st[N];  // 存储每个点的最短路是否已经确定

// 求1号点到n号点的最短路，如果不存在则返回-1
int dijkstra() {
  memset(dist, 0x3f, sizeof dist);
  dist[1] = 0;

  for (int i = 0; i < n - 1; i++) {
    int t = -1; // 在还未确定最短路的点中，寻找距离最小的点
    for (int j = 1; j ≤ n; j++)
      if (!st[j] && (t == -1 || dist[t] > dist[j]))
        t = j;

    // 用t更新其他点的距离
    for (int j = 1; j ≤ n; j++)
      dist[j] = min(dist[j], dist[t] + g[t][j]);

    st[t] = true;
  }

  if (dist[n] == 0x3f3f3f3f)
    return -1;
  return dist[n];
}
```

临接表模板

```
#include <bits/stdc++.h>
using namespace std;
const int N = 1e4 + 5; // 最大点数
const int M = 5e5 + 5; // 最大边数
// 邻接表
int head[N], ver[M];  // 表头 边的终点
int edge[M], Next[M]; // 边的权值 下一条边的编号
int idx = 0;
void add(int u, int v, int w) {
  ver[idx] = v;
  edge[idx] = w;
  Next[idx] = head[u];
  head[u] = idx++;
}
// Dijkstra
int dis[N], parent[N]; // 距离数组 父节点
bool vis[N];           // 是否已经是最短路径
int main() {
```

```
  // 初始化
  memset(head, -1, sizeof head);
  memset(dis, 0x3f, sizeof dis);
  memset(vis, false, sizeof vis);
  // 输入
  int n, m, s; // 点数 边数 起点
  cin >> n >> m >> s;
  for (int i = 0; i < m; i++) {
    int u, v, w;
    cin >> u >> v >> w;
    add(u, v, w);
  }
  // Dijkstra
  dis[s] = 0;
  for (int i = 0; i < n - 1; i++) { // 循环到n-1即可
    // 1、找到vis为false，距离dis最小的点
    int t = -1;
    for (int j = 1; j <= n; j++)
      if (vis[j] == false && (t == -1 || dis[j] < dis[t]))
        t = j;
    // 2、记录点t已经是最短路
    if (t == -1)
      break;
    vis[t] = true;
    // 3、用点t更新其他点的距离
    int it = head[t];
    while (it != -1) {
      int nx = ver[it]; // 由点t为起点的边下一个点
      dis[nx] = min(dis[nx], dis[t] + edge[it]);
      it = Next[it];
    }
  }
  // 输出答案
  for (int i = 1; i <= n; i++) {
    if (dis[i] != 0x3f3f3f3f)
      cout << dis[i] << ' ';
    else
      cout << INT_MAX << ' ';
  }
  cout << endl;
}
```

7.2.2. 堆优化 Dijkstra

临接表模板

```
int n;                              // 点的数量
int h[N], w[N], e[N], ne[N], idx = 0; // 邻接表存储所有边
int dist[N];                        // 存储所有点到1号点的距离
bool st[N]; // 存储每个点的最短距离是否已确定

// 求1号点到n号点的最短距离，如果不存在，则返回-1
int dijkstra() {
  // 初始化 dis 数组全为正无限，初始化起点的 dis 为 0
  memset(dist, 0x3f, sizeof dist);
  dist[1] = 0;
```

```
  priority_queue<PII, vector<PII>, greater<PII>> heap;
  heap.push({0, 1}); // first存储距离，second存储节点编号

  while (heap.size()) {
    // 寻找未确定最短路的点中的 dis 最小的点，记为 t
    auto t = heap.top();
    heap.pop();
    int ver = t.second, distance = t.first;
    // 如果 t 的最短路已经确定，则跳过
    if (st[ver])
      continue;
    // 将 t 的最短路设为已确定
    st[ver] = true;
    // 更新 t 的所有出边
    for (int i = h[ver]; i != -1; i = ne[i]) {
      int j = e[i];
      if (dist[j] > distance + w[i]) {
        dist[j] = distance + w[i];
        heap.push({dist[j], j});
      }
    }
  }

  if (dist[n] == 0x3f3f3f3f)
    return -1;
  return dist[n];
}
```

7.2.3. Bellman-Ford

```
int n, m;    // n表示点数，m表示边数
int dist[N]; // dist[x]存储1到x的最短路距离

struct Edge // 边，a表示出点，b表示入点，w表示边的权重
{
  int a, b, w;
} edges[M];

// 求1到n的最短路距离，如果无法从1走到n，则返回-1。
int bellman_ford() {
  // 初始化所有的 dis 为无限大，设置起点 dis 为 0
  memset(dist, 0x3f, sizeof dist);
  dist[1] = 0;
  // 循环 n 次
  for (int i = 0; i < n; i++) {
    // 每次遍历所有的边
    for (int j = 0; j < m; j++) {
      // 松弛边
      int a = edges[j].a, b = edges[j].b, w = edges[j].w;
      if (dist[b] > dist[a] + w)
        dist[b] = dist[a] + w;
    }
  }
  if (dist[n] > 0x3f3f3f3f / 2)
    return -1;
```

```
  return dist[n];
}
// 如果第n次迭代仍然会松弛三角不等式，就说明存在一条长度是n+1的最短路径，图中存在负权回路
```

7.2.4. SPFA

```
int n;                              // 总点数
int h[N], w[N], e[N], ne[N], idx; // 邻接表存储所有边
int dist[N];                        // 存储每个点到1号点的最短距离
bool st[N];                         // 存储每个点是否在队列中
// 求1号点到n号点的最短路距离，如果从1号点无法走到n号点则返回-1
int spfa() {
  // 初始化 dis 数组全为正无限，初始化起点的 dis 为 0
  memset(dist, 0x3f, sizeof dist);
  dist[1] = 0;
  // 起点入队
  queue<int> q;
  q.push(1);
  st[1] = true;
  while (q.size()) {
    // 将队头点 t 取出
    auto t = q.front();
    q.pop();
    st[t] = false;
    // 更新 t 的所有出边，如果有点被更新就入队
    for (int i = h[t]; i != -1; i = ne[i]) {
      int j = e[i];
      if (dist[j] > dist[t] + w[i]) {
        dist[j] = dist[t] + w[i];
        if (!st[j]) // 如果队列中已存在j，则不需要将j重复插入
        {
          q.push(j);
          st[j] = true;
        }
      }
    }
  }
  if (dist[n] == 0x3f3f3f3f)
    return -1;
  return dist[n];
}
```

7.2.5. Floyd-Warshall

```
// 初始化:
for (int i = 1; i ≤ n; i++)
  for (int j = 1; j ≤ n; j++)
    if (i == j) d[i][j] = 0;
    else d[i][j] = INF;
/* 输入 */
// 算法结束后，d[a][b]表示a到b的最短距离
void floyd() {
  for (int k = 1; k ≤ n; k++)
    for (int i = 1; i ≤ n; i++)
      for (int j = 1; j ≤ n; j++)
```

```
        d[i][j] = min(d[i][j], d[i][k] + d[k][j]);
}
```

7.3. 最小生成树

- 稠密图：朴素 Prim $O(n^2)$
- 稀疏图：堆优化 Prim $O(m \log n)$
- 稀疏图常用：Kruskal $O(m \log n)$

7.3.1. Prim

```
int n;       // n表示点数
int g[N][N]; // 邻接矩阵，存储所有边
int dist[N]; // 存储其他点到当前最小生成树的距离
bool st[N];  // 存储每个点是否已经在生成树中

// 如果图不连通，则返回INF(值是0x3f3f3f3f)，否则返回最小生成树的树边权重之和
int prim() {
  // 初始化 dist 为正无限
  memset(dist, 0x3f, sizeof dist);
  // 循环 n 次（加点 n 次）
  int res = 0;
  for (int i = 0; i < n; i++) {
    // 找到离集合最近的点 t（ dis最小 ）
    int t = -1;
    for (int j = 1; j <= n; j++)
      if (!st[j] && (t == -1 || dist[t] > dist[j]))
        t = j;
    // 如果最近点 t 的距离为正无限，则说明图不连通
    if (i && dist[t] == INF)
      return INF;
    // 如果 t 是第一个点，则不需要加上它的距离
    if (i)
      res += dist[t];
    // 标记点 t 已经在树中
    st[t] = true;
    // 用 t 更新其他点的距离
    for (int j = 1; j <= n; j++)
      dist[j] = min(dist[j], g[t][j]);
  }
  return res;
}
```

7.3.2. Kruskal

```
int n, m; // n是点数，m是边数
int p[N]; // 并查集的父节点数组
struct Edge // 存储边
{
  int a, b, w;
  bool operator<(const Edge &W) const { return w < W.w; }
} edges[M];
int find(int x) // 并查集
{
  if (p[x] != x) p[x] = find(p[x]);
  return p[x];
```

```
}
int kruskal() {
  // 按权排序所有边
  sort(edges, edges + m);
  // 初始化并查集
  for (int i = 1; i ≤ n; i++)
    p[i] = i;
  // 枚举每条边
  int res = 0, cnt = 0;
  for (int i = 0; i < m; i++) {
    // 对于边a→b,如果a和b不在同一集合中,则加入边
    int a = edges[i].a, b = edges[i].b, w = edges[i].w;
    a = find(a), b = find(b);
    if (a ≠ b) {
      p[a] = b;
      res += w;
      cnt++;
    }
  }
  if (cnt < n - 1)
    return INF;
  return res;
}
```

7.4. 二分图

- 染色法 $O(n+m)$
- 匈牙利算法 $O(nm)$ （实际运行远小于 $O(nm)$）

7.4.1. 染色法

```
int n;                          // n表示点数
int h[N], e[M], ne[M], idx; // 邻接表存储图
int color[N]; // 表示每个点的颜色, -1表示未染色, 0表示白色, 1表示黑色

// 参数: u表示当前节点, c表示当前点的颜色
bool dfs(int u, int c) {
  color[u] = c;
  // 遍历 u 的所有出边, 如果未染色则染相反色, 如果已染色则判断是否矛盾
  for (int i = h[u]; i ≠ -1; i = ne[i]) {
    int j = e[i];
    if (color[j] == -1) {
      if (!dfs(j, !c))
        return false;
    } else if (color[j] == c)
      return false;
  }
  return true;
}

bool check() {
  memset(color, -1, sizeof color);
  bool flag = true;
  for (int i = 1; i ≤ n; i++)
    if (color[i] == -1)  // 点 i 未染色
```

```
      if (!dfs(i, 0)) {  // 染色为 0
        flag = false;    // 染色失败说明不是二分图
        break;
      }
  return flag;
}
```

7.4.2. 匈牙利算法

```
int n1, n2; // n1表示第一个集合中的点数，n2表示第二个集合中的点数
int h[N], e[M], ne[M], idx; // 邻接表存储所有边，匈牙利算法中只会用到从第一个集合指向第二个集合的边，所以这里只用存一个方向的边
int match[N]; // 存储第二个集合中的每个点当前匹配的第一个集合中的点是哪个
bool st[N];   // 表示第二个集合中的每个点是否已经被遍历过

bool find(int x) {
  for (int i = h[x]; i != -1; i = ne[i]) {
    int j = e[i];
    if (!st[j]) {
      st[j] = true;
      if (match[j] == 0 || find(match[j])) {
        match[j] = x;
        return true;
      }
    }
  }
  return false;
}
// 求最大匹配数，依次枚举第一个集合中的每个点能否匹配第二个集合中的点
int res = 0;
for (int i = 1; i <= n1; i++) {
  memset(st, false, sizeof st);
  if (find(i)) res++;
}
```

7.5. 联通性

7.5.1. Tarjan

```
int h[N], e[M], ne[M], idx;     // 链式前向星
// dfn 表示每个点的访问时间戳，low 表示每个点能访问到的最小时间戳
int dfn[N], low[N], timestamp;
int stk[N], top;
bool in_stk[N];
int id[N], scc_cnt; // 每个点所属分量编号

void tarjan(int u) {
  // 访问当前节点 u，初始化其 dfn 和 low，将其入栈，标记已入栈
  dfn[u] = low[u] = ++timestamp;
  stk[++top] = u, in_stk[u] = true;
  // 遍历 u 的所有邻边 (u, j)
  for (int i = h[u]; ~i; i = ne[i]) {
    int j = e[i];
    if (!dfn[j]) {                      // 如果 j 未访问，则递归访问，访问结束更新 low
      tarjan(j);
      low[u] = min(low[u], low[j]);
```

```
    } else if (in_stk[j])                // 如果 j 已访问，且 j 在栈中，更新 low
      low[u] = min(low[u], dfn[j]);
  }
  // 若 u 是其所在强连通分量的根
  // 循环出栈，标记出栈元素都为同一个强连通分量，直到 u 出栈
  if (dfn[u] == low[u]) {
    ++scc_cnt;
    int y;
    do {
      y = stk[top--];
      in_stk[y] = false;
      id[y] = scc_cnt;
    } while (y != u);
  }
}
```